



University of Essex

School of Computer Science
& Electronics Engineering

End to End Learning of Communication System

Saad Ahsan

Registration number: 2106490

**A THESIS SUBMITTED FOR THE DEGREE OF MASTER
OF SCIENCE IN ELECTRONICS ENGINEERING**

SUPERVISOR: **Dr. Varasteh, Morteza**

AUGUST 25, 2022

Contents

0.1	ABSTRACT	5
1	INTRODUCTION	6
1.1	Motivation and Objective	8
1.2	Dissertation Structure	10
2	BACKGROUND AND LITERATURE	11
2.1	Deep Learning and Physical layer	12
2.2	Deep learning basics	13
2.3	Deep Learning Libraries	14
2.4	Literature Review	15
2.4.1	Deep Learning to Existing Communication System Design	16
2.4.2	End-to-End Communications using Deep Learning	19
3	COMMUNICATION SYSTEMS AS AUTOENCODER	22
3.1	Proposed System Model	23
3.2	System training and testing	27
4	RESULTS AND ANALYSIS	29
4.1	Constellation Diagrams	32
5	CONCLUSION AND FUTURE WORK	36
A	Appendix	43
A.1	Neural Networks	43

List of Figures

2.1	Block structured Communication System	16
3.1	Basic model of communication system	23
3.2	Communication Model	23
3.3	Autoencoder with Multiple layers	25
3.4	Onehot Encoding	25
4.1	BER versus $(\mathbb{E}_B/N_0)$ for the Autoencoder (7,4) and Uncoded BPSK	30
4.2	BER versus $(\mathbb{E}_B/N_0)$ for the Autoencoder (2,2),(8,8) and Uncoded BPSK (2,2),(8,8)	31
4.3	BER versus $(\mathbb{E}_B/N_0)$ for the Autoencoder (2,4)	31
4.4	Constellation diagram for (2,2) configuration	33
4.5	Constellation diagram for (2,4) configuration	33
4.6	Constellation diagram for (2,4) configuration	34
4.7	Constellation diagram for (7,4) configuration	34
4.8	Constellation diagram for (8,8) configuration	34
A.1	Fully Connected Feed Forward Neural Network	46
A.2	Gradient Decent technique	47
A.3	Autoencoder	48

List of Tables

3.1	Autoencoder Summary	26
3.2	list of parameters	27
A.1	Types of Layers of Neural networks	44
A.2	Types of Activation Functions	44

0.1 ABSTRACT

The word communication system brings the idea of complex block structured architecture, where each block is responsible for some complex task. In the scope of this work, we have introduced a novel concept of deep learning for the physical layer. We come up with an entirely new way of looking at the design of communication systems that aims to jointly optimize transmitter, channel, and receiver as a reconstruction task. A deep learning-based autoencoder architecture has been designed to compete with current communication system architectures by simulating the communication system for a single user over the AWGN channel. The proposed autoencoder architecture is trained in four alternative configurations that differ in the number of bits per message symbol ($\mathbf{k}$), communication rate($\mathbf{R}$), and length of message space ($\mathbf{M}$). The training of four distinct configurations is done over the same characteristics of the AWGN channel, and their BER performance is compared with the corresponding traditional communication system architectures. Compared to uncoded BPSK, the suggested autoencoder architecture has the same or better bit error rate (BER) for the same parameters and channel conditions. It has been shown that the difference between the performance of autoencoder and uncoded BPSK rises further as the number of channels($\mathbf{n}$) and length of message space($\mathbf{M}$) increases. The suggested architecture highlights its flexibility and adaptability to changing channel conditions by performing well over the range of channel noise variance. Although, the model was trained for the fixed degree of channel noise variation. After comparing the BER performances of the two systems, Our study provides insights into constellation diagrams, a complex encoded representation of all the messages in the message space. The intrinsic parallel processing architecture, the flexibility of the structure, and increased learning capacity all contribute to the improved BER performance, lower processing complexity, and lower latency of the systems based on autoencoders that demonstrate their usefulness and viability as an alternative to traditional communication architectures.

INTRODUCTION

In today's computerized world, which has altered how we live, work, and connect, wireless networks and related services have emerged as crucial building blocks. The emergence of numerous novel services and applications, such as autonomous vehicles, remote medical diagnostics and surgeries, smart factories and cities, and personal assistants based in the field of artificial intelligence and machine learning, are posing new challenges to the existing communication mechanisms and methods in terms of latency, reliability, energy efficiency, flexibility, and connection density. In order to meet the demanding requirements of those many verticals, there is a growing need for wireless system studies to create novel architectures, methodologies, and algorithms in almost every level of a communications system. Research efforts are ongoing in the telecommunications sector, intending to modernize all currently existing wireless applications.

One of the examples of continuous telecommunications research activities is the new developing mobile communication technology, which strives to overcome the previously stated communication difficulties and has the strictest standards for its use. For instance, the magnetic component of this mobile communication technology's evolution, which may also be the most difficult, is namely, Ultra reliable low latency communication, which must satisfy two demanding and incompatible standards, low latency and ultra-high reliability. If we're talking about numbers and facts, it aims to provide very high reliability with a bit error rate (BER) of roughly 10^{-5} in 1 milliseconds and an end-to-end delay of about 10 milliseconds[14]. High reliability means that channel estimation accuracy should be improved to minimize any

loss that may be brought on by the channel estimation process. It will be done using advanced channel estimation methods and by providing the pilots with more resources, which again raises concerns about the latency specifications because adding more pilots increases the amount of control overhead, which reduces communication throughput and, ultimately causes delays. Signal processing at the transmitter and receiver without unnecessary delays is another issue to be concerned about to meet the Ultra-Reliable Low-latency Communication system's low latency criteria. Therefore, all of these considerations are necessary for the successful implementation of the Ultra-Reliable Low-latency Communication systems, which necessitates the use of unique architectures, strategies, and algorithms at nearly every layer of the communication system.

Complex mathematical modelling, alongside the knowledge of statistical and information theories, are the foundations of communication systems, allowing for the simulation of channels [23], and designing of optimal signalling and detection protocols for safe data transmissions that compensate for various hardware limitations of the physical layer [28]. Current traditional communication theories, however, highlight several inherent challenges in exchanging massive amounts of data at super high speeds, such as channel modelling and efficient signal processing with minimum possible delays.. Due to the difficulty of implementing a reliable mathematical method over the conventional system to simulate various circumstances, the performance of the traditional block structured communication system ranks poorly in some scenarios, such as in the fields of computer vision and natural language processing, etc. So there comes room for implementing deep learning and machine learning in the communication system. Recent interest in using deep learning algorithms for physical layer implementations of communication systems has increased due to several benefits they possess. This has led to a reconsideration of the design problem for communications systems and the replacement of traditional block-structured communication systems with some deep learning models.

1.1 Motivation and Objective

To reliably send a message from a device to another device over a channel using a transmitter and a receiver is a fundamental need of a communications system. The traditional architecture of a communication system is decentralised, with separate modules handling various stages of the process, such as source coding, encryption, modulation, decryption, demodulation, etc. Such a modular design makes it easier to examine, optimise, and exert command over each component's specific set of functions. However, it is not always apparent that using individually optimised processing blocks would result in the best end-to-end performance, and this approach might wind up being sub-optimal sometimes [21]. On the other hand, the idea of a deep learning-based communication system enters the picture[21]. It adheres to the original concept of the communication system. It seeks to jointly optimize the transmitter and receiver in an end-to-end manner without using the communication system's block structured model mentioned previously[19]. The proposed deep learning model appears more attractive for practical implementations because it is simple, less complex, straightforward, requires less computational resources, and owes to less power consumption and processing delays for an equal or better performance compared to existing conventional communication systems. Based on the attributes mentioned above of the deep learning model, there is a large room for applying these models in the communication sector. In this work, we are designing a deep neural network(DNN) based model for communication to achieve end-to-end learning of communication systems to compare its performance with the current state-of-the-art technologies[21]. The performance of the designed model is measured in terms of bit error rate(BER). The system is optimized to achieve better or equal performance compared to currently employed models and additionally requires less computational power and overcoming latency constraints.

Motivated By previous research and work done in achieving end-to-end learning of communication systems using Deep learning models. A feed-forward neural network is designed to simulate the transmitter, channel, and receiver, and the overall system is given the name of auto-encoder[21]. As the name suggests, an autoencoder is a neural network that learns and adapts to compress and decompress the data at the transmitter and receiver end, respectively, in the presence of provided channel model. The target of the autoencoder is to deliver its input at the output with minimum possible reconstruction error. The idea

of deep neural network-based autoencoder belongs to unsupervised learning and differs in overall structure and modulation techniques adapted by conventional communication system models. With its performance, it competes with the performance of traditional models while requiring fewer resources and complexities. Researchers made efforts in the field of machine learning to achieve end-to-end performance based on a need to improve or entirely substitute the current communication models since the massive size of datasets and latency constraint and the existing structure of communication model does not allow effective implementation of machine learning algorithms. This work aims to study and examine physical layer needs and requirements in the literature to better tackle the complexities faced by existing technologies and why they are falling behind. Moreover, to understand the severity of the problem and why there is a need for a replacement. Since deep learning models for communication are prioritized in this work as a replacement, we also carefully examine the deep learning models for communication in the literature and how their approaches to the physical layer differ from existing models. Hence, concluding why they are superior to existing models and how they might outperform them. As previously indicated, in this work, we adopt a deep learning-based model of communication to mimic the communication system, Which allowed us to learn how to simulate a communication system with the help of deep neural networks in an autoencoder configuration. Additionally, we discover how to configure an autoencoder to achieve an end-to-end performance of a communication system over a noisy AWGN channel, emulated by including gaussian noise as an additional channel noise. After successfully using an autoencoder to simulate a communication system and achieve end-to-end learning, the system's effectiveness is evaluated compared to uncoded communication systems already in use.

1.2 Dissertation Structure

There are five chapters in the thesis. This chapter outlines the constraints and difficulties that future communications systems will need to overcome, plus our justifications for looking into deep learning-based methods for the physical layer of the communication system. The background and literature for the thesis project are detailed in the following chapter. There, we go into great length on the possibilities of deep learning for the physical layer as well as some fundamentals of deep learning that are pertinent to the scope of this work in the literature and the research study that follows covers the existing research on deep learning-based end-to-end communications as well as deep learning-based block structured communications. Following this, we address the autoencoder concept for end-to-end learning of communications systems and evaluate the suggested autoencoder-based end-to-end system in comparison to traditional uncoded systems across a range of modulation schemes.[\[21\]](#). Concerning the work completed in the previous section, we provide the results and constellation diagrams in the fourth chapter. In the end, investigation results are summarised in the last chapter, and future potential for the work is concluded.

BACKGROUND AND LITERATURE

Machine learning algorithms have gained popularity in every aspect of life. One of the most well-known applications comes in the field of mobile communication, where machine learning algorithms have been tested to provide a variety of services in the physical layer of the communication system, including encoding and decoding[16], channel modelling and identification[29], modulation recognition [11] [27], channel estimation and equalization[15] [5]. However, it is observed that machine learning algorithms have not come to play on commercial grounds because their learning ability is not sufficient enough to manage processing challenges posed by interacting with the physical channel. Hence, this results in poor performance. The performance gap between employed machine learning algorithms and complex physical channels is bridged by the introduction of deep learning(DL). Deep learning (DL) is thought to have the potential to significantly improve performance as compared with existing machine learning algorithms by removing the barriers imposed by traditional ML algorithms, such as feature extraction and structural flexibility. It introduces the concept of deep modularization, which significantly enhances feature extraction[26]. In particular, instead of manually extracting features, deep learning-based algorithms might be employed to learn features from raw data. Additionally, it allows flexible model structure, which can be set via hyperparameter tuning to achieve end-to-end learning performance.

2.1 Deep Learning and Physical layer

The idea of deep learning for the physical layer has been developing for a while, and many researchers are now interested in it. Significant improvements and introduction of new structural models opened new ventures in wireless communication. In this section, we highlight how deep learning models aim to cater to the limitations faced by the physical layer of communication systems, followed by deep learning basics and libraries involved in the implementation of deep learning models. At last, we throw some light on the already work done in the field of deep learning concerning the physical layer of communication systems.

Conventional communication systems make use of the various signal processing algorithms based on the foundations of information theory and statistics. However, these models are proved to be the best fit for mathematical models. Still, when it comes to practical implementations, it may be challenging to adequately capture all of the flaws and non-linearities present in real applications. On the other hand, comes the deep learning models, which can simulate the communication system despite complex channel conditions and can also take care of the hardware flaws, which appears to be a tremendous challenge for mathematical models[21][4]. Secondly, deep learning models are trained to achieve end-to-end performance by optimizing the parameters involved to attain the minimum possible loss. Values of these optimized parameters, known as weights, represent a learned algorithm instead of requiring a well-defined mathematical model[26]. Furthermore, the unique, distributed, and parallel processing architecture of deep learning models offer to process massive datasets with promising computational speeds and processing capabilities, which demonstrates how the deep learning model's parallelized architectures can considerably boost computational throughput[21][26]. Along with high computational speeds and the ability to deal with a massive amount of data, distributed memory parallel processing architectures, such as graphical processing units (GPUs), have been shown to be very energy efficient and capable of offering significant computing efficiency. Deep learning-based models appear as a substitute to existing block structured communication models because deep learning models focus on optimization of end-to-end performance rather than optimizing each block of communication model separately to achieve optimum performance[21], and if such end-to-end architecture is implemented for a communication system, there is a further room improve the overall performance. Additionally, the use of deep learning techniques in a wide range of applications

is made possible by the tools, frameworks, and libraries[3] [22] [12] that have been developed for testing and modelling deep learning models in more convenient and straightforward ways.

2.2 Deep learning basics

Deep Learning is a sub-branch of machine learning which expands to the areas of supervised, unsupervised, and reinforcement learning algorithms to support the tasks such as prediction of continuous data, classification and regression problems and much more, which is supported by machine learning algorithms. Whereas, unlike machine learning algorithms, deep learning models offer multiple layered structures and automatic feature extracting techniques that extract the features from raw data, using processing units or mathematical functions involved in the hidden structure of the deep learning models to achieve the best possible performance. When we talk about the layered structure, the first thing came to mind is deep learning models. Further, in deep learning models, the most famous layered structures are deep random forests, neural processes, Gaussian processes, and Deep neural networks(DNN)[31]. However, these are some commonly used and famous multiple layer structured deep learning models. Still, among them, the deep learning model, which has gained immense popularity due to its versatile performance and robust structure, is considered to be a Deep neural network. The work on neural networks started in the late '50s of the last century when for the first time[24], concepts of biological learning evolved, and as a result, a single neuron was trained, and the model was given the name perceptron[10]. However, after two decades, scientists discovered that it is possible to train a neural network with multiple hidden layers, and the process involved in the training of such a network was given the name of back-propagation[31]. Afterwards, the scientist's interest increased in the field of deep learning. Moreover, the technological advancements and massive data-set sizes have further promoted the research in the field of deep learning, allowing deep learning-based solutions in broad areas of application[31]. In this work, our main focus is on the neural network-based deep learning model. In Appendix[5], we will discuss some of the common architectures of neural networks which are used in the scope of this work.

2.3 Deep Learning Libraries

As mentioned before, In recent years, deep learning models have experienced immense growth and popularity for their application in different aspects such as speech recognition, image and video recognition, natural language processing, etc. The significant increase in interest in deep learning models has led to the development of numerous tools, dedicated libraries, and frameworks that further promote the practical usage of deep learning models and allow the building and training of models more efficiently. Otherwise, defining forward-ing behaviours, gradient propagation activities at each layer, and implementing effective and quick optimization methods for model training are all complicated tasks. Moreover, most of these tools are developed to support multiple programming languages and enables the backend computations such as automatic differentiation of training loss functions for loss optimization and process like normalization[21], etc. Some of the commonly used deep learning libraries are listed below:

- **Tensor flow**

Tensor flow is an open-source library developed back in 2015 to support numerical computations and machine learning[3]. In TensorFlow, computation, shared state, and operations that modify that state is represented by dataflow graphs. Those dataflow graphs are shared among multiple devices in a cluster and various processing units within the same device, such as multicore central processing units (CPUs), all-purpose graphics processing units (GPUs), and specially created application-specific integrated circuits (ASICs) known as Tensor processing units (TPUs)[3]. A user can use a programming language of their own choice out of the programming languages supported by tensor, namely, Python, C++, Java, Go, and R. Whereas python is the most frequently adopted programming language for building deep learning models. Some of the specific deep learning toolboxes based on TensorFlow that offer higher-level programming interfaces are Keras[6], Luminoth, and TensorLayer. In this work, we are mainly using Keras to build a neural network in an autoencoder configuration. It offers a versatile interface that facilitates easy building and modification of deep learning models.

- **PyTorch**

PyTorch is also one of the open source libraries for deep learning. It is a python based

library that has the base foundation of the Torch library. Torch is also an open-source machine learning library. Although python is the primary focus of development in PyTorch, it also has a c++ interface. PyTorch is considered one of the most popular deep learning research platforms yet has the stability and support required for production deployment[22]. It is renowned for offering two of the most advanced features, including tensor calculations with robust GPU acceleration and the construction of deep neural networks on tape-based autograd(automatic differentiation) systems intended for quick and Python-like execution. Since it's simple to create NNs with PyTorch, it's becoming more and more attractive to users.

- **Caffe**

Convolutional Architecture for Fast Feature Embedding, or Caffe, is a deep learning framework that was created by the University of California, Berkeley in 2014. Although it was made in C++, a Python interface is also provided. Caffe is renowned for its expressive architecture, which supports alternate CPU and GPU configurations, deep learning model training on multiple GPUs inside distributed systems, and mobile device implementation. Caffe also supports libraries like NVIDIA cuDNN and Intel MKL, which are recognized as GPU- and CPU-based acceleration computational kernel libraries[12]. Second, Caffe's speed makes it the perfect choice for usage in academic projects[20], commercial applications, and even extensive industrial applications in vision, speech recognition, and multimedia.

2.4 Literature Review

The idea of deep learning for the physical layer took shape a long time ago, increasing the interest of researchers. Some efforts have been made to put deep learning into action for communication systems using various deep learning designs like as feed-forward neural networks, convolutional neural networks, and Autoencoder architecture due to the promise of deep learning for the physical layer,[21] etc. In literature, there are two main strategies adopted either the work is done on already in use conventional block structured models by introducing deep learning to it, or a revolutionary work is done to eliminate the conventional block structured communication system and propose deep learning based models instead, which aims to optimize end-to-end performance. Introducing deep learning concepts to

existing architecture seeks to enhance its performance by optimizing each block separately. Below, we highlight some of the work done in deep learning for communication systems that support the aforementioned two approaches.

2.4.1 Deep Learning to Existing Communication System Design

Communication systems are essential to our daily lives as they link between two distant users, enabling them to send and receive information in a trusted environment. A good communication system maintains the integrity of the information transferred with the minimum possible delays, which means that the sent information from the sender should be securely received by the receiver without being information corrupted, and all this process should take place in the minimum possible time. The process of sending and receiving information is not as simple as it looks likes. To do so, there is a complex architecture of communication system which is based on different blocks, each performing a specified task as shown in the figure below:

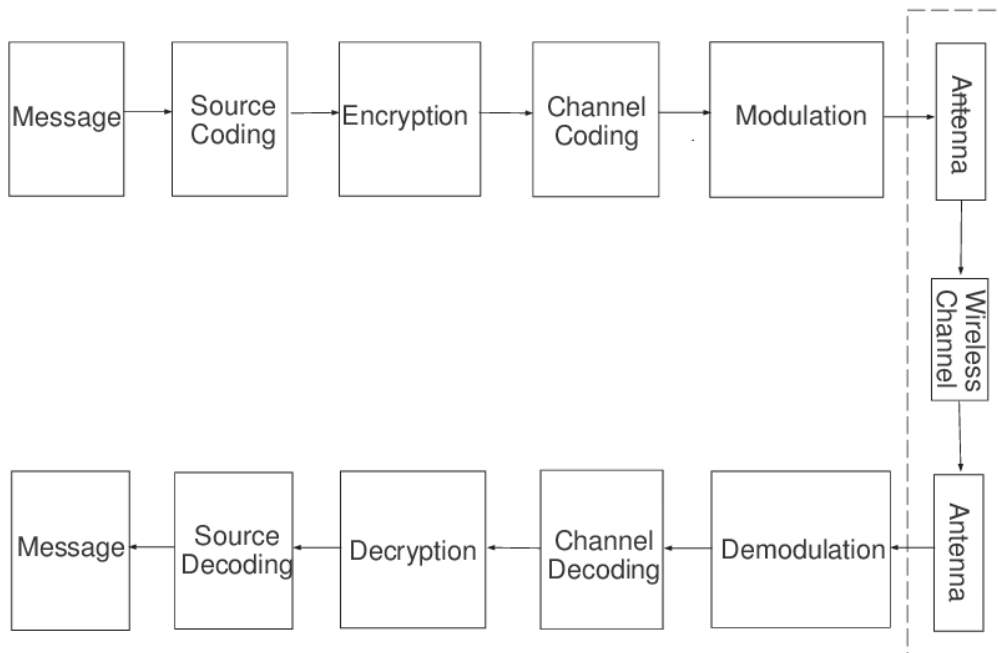


Figure 2.1: Block structured Communication System

As seen in the figure above, each block in a communication system performs a specific job. To obtain the best possible overall performance, each block's performance is enhanced by deploying expert knowledge algorithms that were created decades ago. Over the years, studies have suggested the involvement of machine learning and deep learning models

such as support vector machines (SVM), feed-forward neural networks, and deep learning models have been used as a substitute to conventional algorithms deployed in each block of a communication system such as channel estimation and detection[9] [30] [18], modulation recognition[25], channel encoding, and decoding[26] [13]. These deep learning approaches are believed to outperform conventional algorithms, especially in complex channel conditions, due to the effective learning capabilities of the deep learning algorithms. Moreover, deep learning models impose less computational complexities, power consumption, and processing delays. Below, we will discuss the application of deep learning models to some of the blocks of conventional communication systems.

- **Joint Channel Estimation and Signal Detection using Deep Learning**

In this literature work[30], Deep learning-based models have been employed for channel estimation and signal detection over an OFDM (Orthogonal frequency division multiplexing) system, which aims to cater to both processing blocks, channel estimation, and signal detection, jointly with the aid of deep learning based architecture to handle wireless OFDM channels in an end-to-end manner. Whereas in traditional communication systems, the receiver's channel estimate and signal detection are treated as independent blocks. Prior to the detection of sent information, channel state information (CSI) is calculated explicitly using pilots. The estimated CSI is then utilised to recover the transmitted information at the receiver. In a deep learning-based approach, a fully connected deep neural network is implemented, which aims to estimate CSI implicitly and recover transmitted symbols directly at the same time. First, the built model is trained over artificially generated channel noise based on channel statistics, so when using the model over the OFDM channel for data transmission, it can deal with added channel noise. When the proposed model is tested over the OFDM channel, its performance shows that the proposed deep learning-based model is robust against channel noise and interference, and it produces results comparable to the MMSE (Minimum Mean Square Error) estimator, and its performance is further enhanced when a fewer number of pilots are used, a CP (cyclic prefix) is omitted, and when nonlinear clipping noise exists[30]. Furthermore, it can be concluded that deep neural networks can remember and analyze complex channel characteristics and are believed to be a promising tool for channel estimation and signal detection over complex wireless channels.

- **Modulation Recognition using Deep Learning**

Modulation recognition is an important intermediate that is performed between signal detection and demodulation. As the name suggests, this step is performed to recognize the different types of modulations the received signal is subjected to, and it is a vital process in communication that allows the interpretation of the received signal in an effective manner. According to published studies, several scientists have been interested in modulation recognition over time, and it turns out that several distinct methods were used for implementation, such as support vector machines, random forests, and feed-forward neural networks. However, these applied methods possess some common procedures, namely, classification, feature extraction, and preprocessing techniques. Furthermore, Some studies have shown sophisticated methods that integrate Neural Network based classification with pattern recognition[17] [7] on expert feature maps, such as the spectral coherence function or α -profile. Work in [26] suggests a neural network-based architecture for modulation recognition to recognize a noisy band-limited signal which has been passed through thirteen different types of digital and analogue modulation schemes. This method works on an automatic feature extraction technique and fetches the features of the received signal along with the information to distinguish between digital or analogue modulations. After feature extraction, a multi-layer neural network is fed extracted features, including amplitude, phase, and frequency. This neural network recognizes the type of modulation scheme applied. Further, after the recognition of applied modulation, it is fed to another neural network for the estimation of M-ary amplitude shift-keying (MASK) and the M-ary frequency shift keying (MFSK) modulations. After studying this approach, it is evident that results achieved from this work in [26] solely depend upon feature extraction. However, due to the limited scope of conventional neural network learning abilities, some efforts have been made on the deep learning side due to their well-known learning ability to get over the drawbacks of feature-based modulation recognition techniques.

The authors of[21] suggest three competitive approaches for modulation recognition. One is a deep learning-based approach; namely, a Convolutional neural network design for modulation recognition and the other two are extreme gradient boosting with 1000 estimators and a single scikit-learn tree working on the extracted expert features respectively. The suggested deep neural network architecture intends to eliminate the

necessity for feature extraction and instead presents an efficient method of automatic learning of features from the raw data in order to achieve optimum end-to-end learning performance with its well-known learning skills. The proposed convolutional neural network is trained using a dataset of 1.2M sequences of 128 complex-valued basebands in-phase and quadrature (I, Q) samples subjected to 10 different digital and analogue single-carrier modulation schemes and had experienced multiple fading effects along with clock and carrier rate offset. After training, the proposed three approaches are tested on single carrier modulation schemes based on sampled radio frequency time-series data. Obtained results indicate that the CNN-based modulation classifier works better than two other techniques, primarily in the low to medium signal-to-noise ratio (SNR) region. However, in the high SNR range, the performance of CNN and boosted trees are comparable.

2.4.2 End-to-End Communications using Deep Learning

As mentioned before, there are two strategies adopted for the penetration of deep learning concepts into the communication system. One is to introduce deep learning-based models and algorithms to the existing communication system architectures, while the other is to propose an entirely new system architecture that replaces the already in-use block-structured architecture. Above, we have discussed some efforts to involve deep learning-based models in the conventional block structured communication systems, in which either one or two blocks of the communication system are incorporated with deep learning models. However, the implemented deep learning concepts to separate blocks of communication optimize the performance of that specific block, but the original goal of the communication system to achieve end-to-end performance for transmitting data from transmitter to receiver over a noisy wireless channel is still not guaranteed. Based on this reasoning, a unique revolutionary Deep learning-based idea has been proposed in the literature that reframes the concept of communication between two devices as an end-to-end reconstruction job and eliminates the need for existing block structured conventional communication system architectures[21]. This idea comes into shape by simulating communication systems with the help of deep neural networks in auto-encoder configuration. Below is some work found in the literature that aims to discuss the design and operation of autoencoders, as well as the potential autoencoders that have to replace the current communication system architectures.

- **Autoencoder based End-to-End Communications**

As discussed above, the unique idea of replacing the current communication system architecture with an autoencoder architecture based on deep learning is supported by the literature [8][21][19]. A communication task is dealt with as an end-to-end reconstruction task to do away with the limitation of achieving end-to-end optimization. The proposed design optimizes transmitter, channel, and receiver cumulatively under one single model. It models the entire communication system as a fully connected neural network, which aims to reconstruct its input at its output in an optimized manner, and is given the name of the autoencoder. It is shown that deep neural networks have the capability to learn transmitter and receiver implementations over provided channel model, and they can be optimized for the given loss function. In [21], a communication system with AWGN (Additive white Gaussian noise) channel is simulated as a fully connected feed-forward neural network known as autoencoder, which aims to optimize transmitter and receiver implementations to minimize the provided loss function, Bit Error rate (BER). The optimization process is the combined effort of receiver and transmitter which learn through the training process for autoencoder in which message " s " out of " M " possible messages $s \in M = (1, 2, \dots, M)$ is intended to be sent by the transmitter. The sent message is encoded by the transmitter to obtain a complex representation " x ", after which the AWGN channel noise is added by adding a single Gaussian layer with defined noise variance. This complex noise-added encoded representation is fed to the receiver. At the receiver, the received noise added encoded message is decoded in a way to reconstruct the original sent message " s ". The whole network is trained in this way to achieve end-to-end optimized performance for minimizing the BER (Bit Error Rate). In the end, the performance produced by autoencoder-based communication systems is compared with uncoded BPSK (Binary phase shift keying) architecture and it is found that autoencoder has equal performance as compared to applied BPSK modulation, which is clear evidence of end-to-end optimization of the system to minimize the loss function over the given AWGN channel.

Furthermore, in another work [8], the novel concept of autoencoder-based communication systems is expanded towards the implementation for over-the-air transmissions, and a fully connected feed-forward neural network design, identical to previous work

in [21], is adopted along with the use of unsynchronized off-the-shelf software-defined radios (SDRs) and open source Deep learning software libraries, is used to model an entire communication system. The proposed advanced architecture also addresses the limitations faced by the existing architectures, the short block lengths constraint faced by autoencoder models has been overcome by promoting the idea of continuous data transmission. At the same time, the receiver synchronization problem is covered by introducing a frame synchronization module based on another Neural Network. The proposed system architecture produces BLER(Block Error rate) performance competitive to baseline models. However, employing this model across noisy channels with missing gradients still leaves some opportunity for advanced problem-solving, for this reason, a concept of a two-step learning procedure based on transfer learning is introduced which allows the learning process to be carried out again over the real channel so the system can learn all the channel imperfections which were not addressed in the previous training session. Thus, the study has confirmed the viability of actually implementing autoencoder-based communication systems.

After the implementation of the concept of autoencoder-based communication systems for a single transmitter and receiver, the concept is further extended toward multiple transmitters and receivers over the same shared AWGN channel [21]. A straightforward two-user situation is taken into consideration where both transmitters intend to communicate with their corresponding receivers over the same channels at the same time, resulting in interference of both sent messages over the channel. The simple architecture of the feed-forward neural network, same as in the case of a single transmitter and receiver, is adopted with a modification of multiple input and output layers jointly optimized to achieve end-to-end performance over weighted sum of both losses. The results obtained by this multi-user supported architecture are comparable with uncoded QAM (Quadrature amplitude modulation), as the proposed system achieves equivalent or better BLER performance than the uncoded QAM scheme for the same transmission rate. Hence, proving the viability of implementation of autoencoder-based communication systems in multiple-user scenarios.

COMMUNICATION SYSTEMS AS AUTOENCODER

In the previous sections, we have discussed the potential of deep learning models for the physical layer and how the deep learning-based models for communication gained popularity among the research community owing to their simple, versatile architecture and, most importantly, their ability to learn even though in the presence of complex channel conditions and system imperfections. We have seen some examples of how deep learning models have been applied to the design of current communication systems and how autoencoder-based communication systems have helped to shape the idea of end-to-end learning of communication systems. In this section, we will extend the novel concept of end-to-end learning of communication systems and scrutinize how the proposed autoencoder-based communication systems achieve equal or better performance in comparison with existing architectures of the communication system[21]. To comprehend the potential of an autoencoder-based communication system, a design of autoencoder is implemented to simulate the communication system in four different configurations, which differ in communication rates and the number of channels uses, and the performance of the proposed system is compared with the conventional communication system architecture of uncoded BPSK over the AWGN channel when no channel coding is applied. Below we will discuss the system model and develop the autoencoder model for communication as suggested by [21]. The end-to-end optimized system's results are then compared with presently used Uncoded BPSK designs in the following section.

3.1 Proposed System Model

The basic structure of communication system is composed of three components transmitter, receiver and channel of communication as shown in the figure [3.1] below:

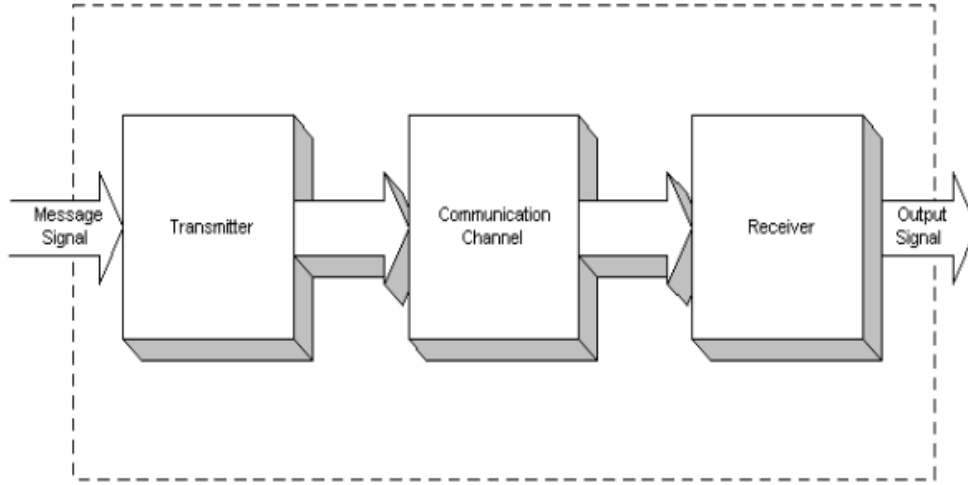


Figure 3.1: Basic model of communication system

Similar kind of communication model is proposed in [21], where the transmitter wants to propagate its message to receiver over the wireless AWGN channel as shown in the figure [3.2] below:

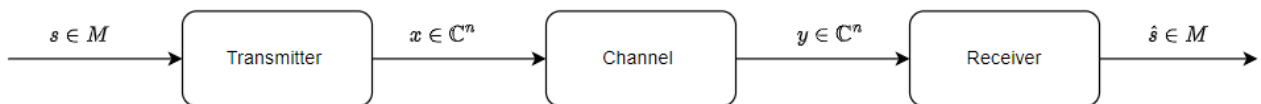


Figure 3.2: Communication Model

In the figure above, "**M**" is the generated message space, all of the messages the transmitter wants to send to the receiver over the given wireless channel. All of the generated messages

in the set " $\mathbf{M}$ " are sent one by one. Whereas " $\mathbf{s}$ " is defined as each single message out of the message space " $\mathbf{M}$ " given by $\mathbf{s} \in \mathbf{M} = (1, 2, \dots, \mathbf{M})$. After passing through the transmitter block, " $\mathbf{s}$ " is converted into " $\mathbf{x}$ ", where " $\mathbf{x}$ " is defined as a complex representation of the input message " $\mathbf{s}$ ". This complex representation is then accumulated with channel characteristics after passing through the channel using " $\mathbf{n}$ " complex symbols and turns out to be " $\mathbf{y}$ ", a complex noise added representation of the input message " $\mathbf{s}$ ". The receiver now receives " $\mathbf{y}$ " and transform it into " $\hat{\mathbf{s}}$ ". Whereas, " $\hat{\mathbf{s}}$ " is defined as the estimation of the originally sent message " $\mathbf{s}$ ". In all this described process for transmitting the message from transmitter to receiver there are some physical constraints imposed by the system in practical scenarios, such as energy constraint imposed by the hardware of the transmitter given by $\|\mathbf{x}\|_2^2 \leq n$, amplitude constraint $|x_i| \leq 1 \forall i$ or an average power constraint $E |x_i|^2 \leq 1 \forall i$. The rate of communication is defined as $R=k/n$, where " k " is the number of bits of each message " $\mathbf{s}$ " and " n " is defined as a number of channel uses. The relation between the number of bits of each message " $\mathbf{s}$ " and the original message space " $\mathbf{M}$ " is given by $k = \log_2(\mathbf{M})$.

The communication system discussed above is built as a deep neural network in an autoencoder configuration. The function of the autoencoder is to first compress its input to a low dimensional vector in some middle layer, as was indicated in the preceding section, and then decompress that low dimensional vector to reconstruct its input at the output with the least amount of error. When using an autoencoder to simulate a communication system, the encoding portion of the device is regarded as the transmitter, the low-dimensional vector as a complex representation to be transmitted through the channel, and the decoding portion as the receiver. The autoencoder architecture adopted in [21] to achieve the goal of end-to-end learning of communication systems is shown below in the figure [3.3]

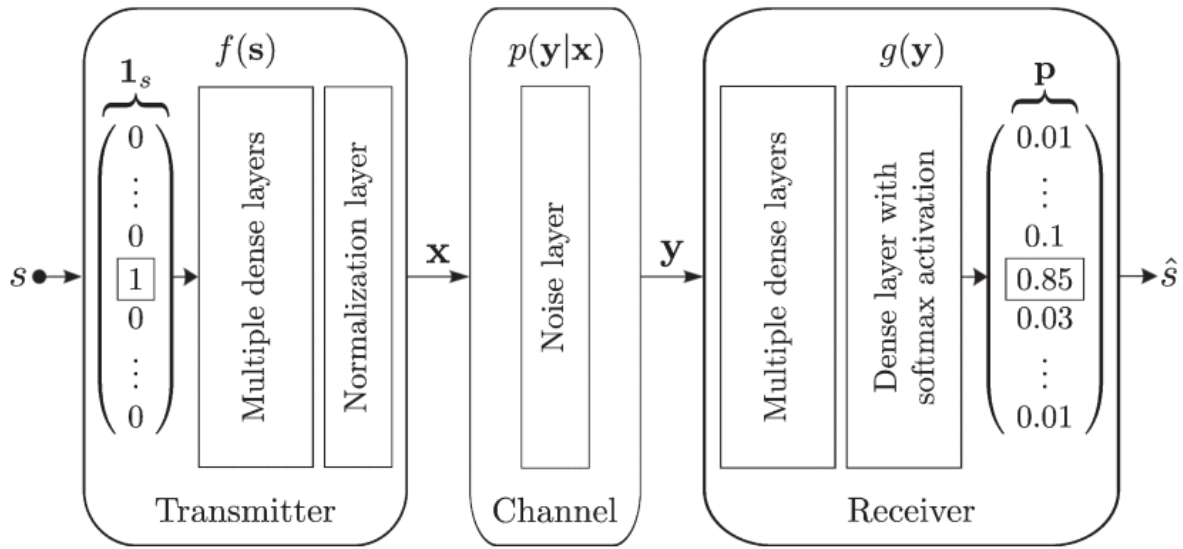


Figure 3.3: Autoencoder with Multiple layers

As it can be seen, the proposed architecture is a fully connected multiple-layer feed-forward neural network has given the name of the autoencoder. The above system is considered as a single unit system instead of a conventional block structured communication system. However, the layers of the proposed neural network are distinguished based on their functionalities. First, three layers are interpreted as the transmitter, including the normalisation layer added at the last of a transmitter, which is added to account for any hardware-imposed physical limitations. The input to the neural network is M-dimensional one-hot encoded vector " $\mathbf{s}$ ", $1_s \in \mathbb{R}$. As mentioned before, message " $\mathbf{s}$ " is the element of generated message space " $\mathbf{M}$ ", one-hot encoding is performed to convert " $\mathbf{s}$ " into a M-dimensional vector. An example of the one-hot encoding can be seen in the figure below:

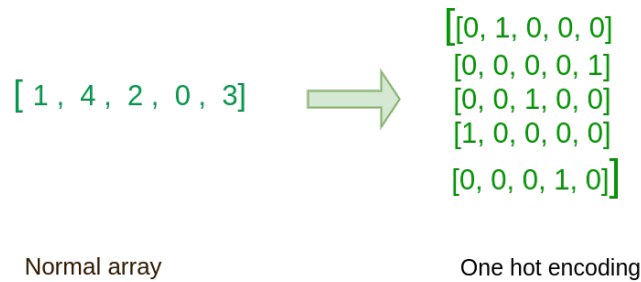


Figure 3.4: Onehot Encoding

Now after one-hot encoding each input message is M-dimensional vector which is fed

to the first dense layer consisting "**M**" output dimensions with "**Relu**" activation function, followed by second dense layer consisting "**n**" output dimensions and "**linear**" activation function, after which here comes the normalisation layer with "**n**" output dimensions, hence the input to the transmitter was "**M**" dimension vector, while output of the transmitter is "**n**" dimensional vector. This means that there "**n**" complex symbols are transmitted through "**n**" channel uses splitting them in real and imaginary parts. The AWGN channel is simulated by adding Gaussian noise with the given noise variance given by $\beta = 1/(2R(\mathbb{E}_B/N_0))$, whereas $(\mathbb{E}_B/N_0)$ is defined as the ratio of energy per bit over added noise power density, in simple words it is **SNR** (Signal to noise ratio). Gaussian noise is added by noise layer which adds a randomly generated vector to its input each time they are called. The output from the added noise layer is also "**n**" dimensional noise added vector now fed to the receiver. The architecture of the receiver is identical to transmitter with first layer as dense layer consisting "**M**" output dimensions with "**Relu**" activation function, followed by second dense layer consisting "**M**" output dimensions and "**Softmax**" activation function. The output of the softmax layer is probabilities of all the possible messages with respect to message space "**M**", $P \in (0,1)^M$. It outputs a "**M**" dimensional vector with probabilities. The index number with the highest probability is known as the reconstructed message $\hat{s}$.

Below is the summary of the designed autoencoder:

Layer	Output Dimensions
Input	M
Dense + Relu	M
Dense + Linear	n
Normalisation	n
Noise	n
Dense + Relu	M
Dense + Softmax	M

Table 3.1: Autoencoder Summary

3.2 System training and testing

The above proposed architecture of autoencoder is implemented for four different configurations that vary in communication rate " $\mathbf{R}$ ", no of bits per message " $\mathbf{k}$ ", no of channel uses " $\mathbf{n}$ " and length of message space " $\mathbf{M}$ " as listed in the table below:

Number of channels ($\mathbf{n}$)	Number of bit per message ($\mathbf{k}$)	Rate of communication ($\mathbf{R}$)	Length of message space ($\mathbf{M}$)
2	2	1	4
2	4	2	16
7	4	$4 \div 7$	16
8	8	1	256

Table 3.2: list of parameters

As mentioned above the proposed autoencoder architecture is trained for different sets of the number of bits per symbol and the number of channel uses[21]. Besides these different values of communication rates " $\mathbf{R}$ " and the lengths of message space " $\mathbf{M}$ ", the selected value of SNR (Signal to noise ratio) is fixed to 7db which turns out to be energy per bit over added noise power density ($\mathbb{E}_B/N_0$) = 5.01187, for the given AWGN channel. Moreover, the model is trained for 100000 examples for each of the four configurations and the Adam optimizer is utilized to achieve end-to-end performance, and its learning rate is set to 0.001. The number of epochs for, $(\mathbf{n}, \mathbf{k}) = (2, 2)$ and $(2, 4)$ configurations were set to 150 with the batch size of 64. Whereas for $(7, 4)$ and $(8, 8)$ the number of epochs was set to 20 with a batch size of 300. However, it was observed that increasing the batch size and reducing the learning rate compensate for to increase in the accuracy of the model. Each of the four configurations listed above is tested following a successful training phase using randomly generated data in the same message space. With 10,000 samples, the first two configurations $(2, 2)$ and $(2, 4)$ are

tested. The remaining two setups, (7,4) and (8,8), are examined using 1500 instances each. In the end, for each of the mentioned configurations constellation diagram is obtained and BER (Bit Error Rate) is plotted for energy per bit over added noise power density (E_B/N_0) in the range from -4 to 12, to compare the performance of the proposed methodology with the baseline models. The following section, displays the obtained scatter plots of the constellation diagrams and compares the obtained BER (Bit Error Rate) performance of autoencoder base communication systems with the baseline system already in use.

RESULTS AND ANALYSIS

The original goal of all the work done was to compare the performance of an end-to-end communications system based on autoencoders to that of a traditional communications system in the absence of channel coding[21]. As a result, we compare the performance of the autoencoder-based system with that of traditional uncoded BPSK over the AWGN channel in this section. We have experimented with a variety of autoencoder designs yielding different communication rates by setting up the appropriate number of channels and number of bits per message. To compare the performance of the autoencoder-based communication system with the traditional uncoded BPSK, Hamming(7,4) with hard decision decoding and Hamming(7,4) with maximum likelihood decoding(MLD) and Bit Error rates(BER) are plotted against energy per bit over added noise power density[21], $(\mathbb{E}_B/N_0)$, as shown in figure [4.1]. It is observed that both systems operate at a communication rate $= 4 \div 7$, and the performance of autoencoder-based communication system in (7,4) configuration, seven-channel uses and 4 bits per symbol, is identical to Hamming (7,4) code with MLD. However, it can be clearly seen from the figure above that the proposed autoencoder (7,4) outperforms uncoded BPSK and Hamming (7,4) with hard decision for the given range of energy per bit over added noise power density $(\mathbb{E}_B/N_0)$. However, it can also be observed that the performance of all the listed models improves over the higher values of $(\mathbb{E}_B/N_0)$.

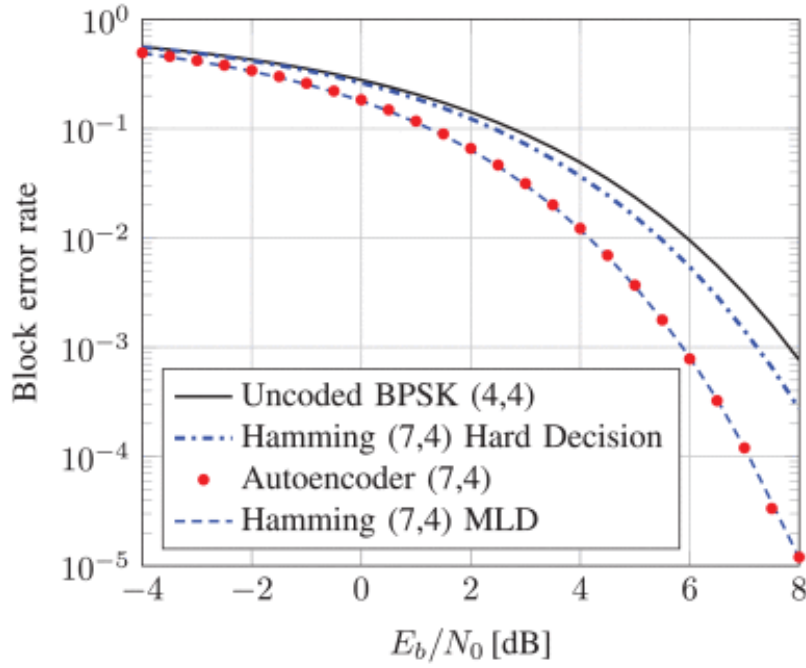


Figure 4.1: BER versus $(\mathbb{E}_B/N_0)$ for the Autoencoder (7,4) and Uncoded BPSK

Moreover, for observing outcomes of communication rate $\mathbf{R}=1$ but for the different number of channel uses, the length of message space $\mathbf{M}$ is set to 4 and 256 for autoencoder configuration (2,2) and (8,8), respectively. The figure [4.2] below displays the BER (Bit Error Rate) performance of the two autoencoder configurations, (2,2) and (8,8), in comparison with equivalent Uncoded BPSK. The graph illustrates the BER performance of four different models, and it is clear from the graph that, when used in the same setting, the autoencoder (2,2) and uncoded BPSK exhibit nearly identical performances. While the performance of an equivalent uncoded BPSK (8,8) is outperformed by autoencoder (8,8), the gap in performance is low for the smaller values of $(\mathbb{E}_B/N_0)$, but as the value of $(\mathbb{E}_B/N_0)$ increases, the gap in performance also increases and autoencoder (8,8) performs better over the higher values of $(\mathbb{E}_B/N_0)$. While the combined plotting of the two architectures of the autoencoder revealed some interesting facts, an increase in length of message space $\mathbf{M}$ along with increasing the number of channels used leads to improve in accuracy; however, the communication rate is the same for both architectures. Since all the channels collaboratively contribute to the transmission of every single message, the more the number of channels, the more the sections of the transmitted message which undergo channel encoding, the splitting of the single

message into more sections makes the message less prone to error hence improving the BER performance.

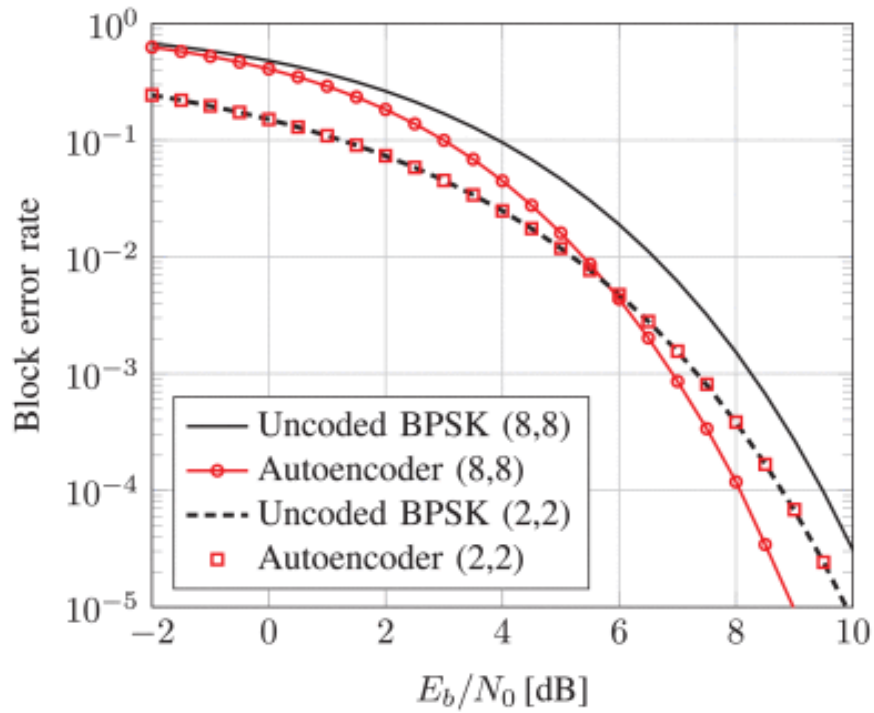


Figure 4.2: BER versus (E_b/N_0) for the Autoencoder (2,2),(8,8) and Uncoded BPSK (2,2),(8,8)

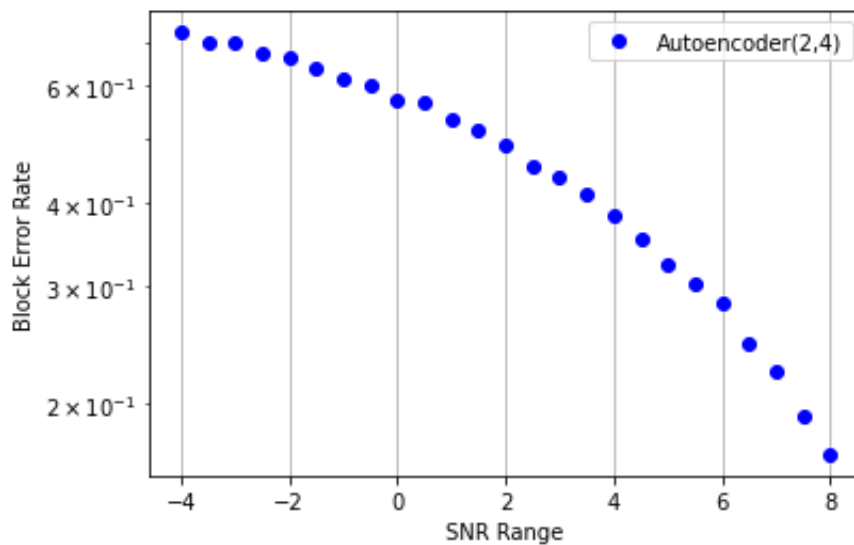


Figure 4.3: BER versus (E_b/N_0) for the Autoencoder (2,4)

After achieving the above results, it is discovered that parameters like $(\mathbb{E}_B/N_0)$, batch size, and the number of epochs play an essential role during the training phase of the model to achieve optimum results. We have experimented with training models with different batch sizes and the number of epochs. However, it was found that an autoencoder with binary channels produces optimum results over a higher number of epochs and smaller batch sizes. On the other hand, for the higher order configurations, it was found that a smaller number of epochs and larger batch sizes improve the performance of the models.

4.1 Constellation Diagrams

Along with comparing BER(Bit Error Rate) performances of the proposed autoencoder architectures with traditional communication system architectures, we had insight constellation diagrams for each configuration as a part of our results. As discussed earlier, the structure of an autoencoder is defined as three main components, transmitter, channel, and receiver, after the transmitter part message is encoded to a complex representation ready for transmission over the given channel. The encoder can produce a symbolic representation for each of the message included in the message space after the model has been trained over the given message space, $\mathbf{M}$. The constellation diagram is a visual representation of the entire encoded representation of the messages in the message space. Each encoded message is composed of two parts. The real part and imaginary part, whereas on the constellation diagram x-axis represent the real part, and the y-axis represents the imaginary part for each point observed on the constellation diagram.

Figure [4.4] refers to the constellation diagram obtained for autoencoder (2,2) with $\mathbf{M}=4$ with a rate of communication $\mathbf{R}=1$, the observed constellation is similar to QPSK(Quadrature phase Shift Keying) like constellation as seen below. Followed by figure [4.5] and figure [4.6] show the constellation representations for autoencoder in (2,4) configurations. The only difference between the configurations is of applied physical constraint to cater hardware imperfections of the transmitter. The constellation observed in figure [4.5] refers to autoencoder (2,4), but to meet energy constraints, it resembles to a rotated 16-PSK (Phase Shift Keying)[21]. Whereas, figure [4.6] also represents constellation diagram observed for autoencoder (2,4) architecture, but now instead of energy constraint, average power constraint is imposed by the transmitter that leads to a mix to mixed pentagonal/hexagonal arrangement. The

constellation is observed to have one symbol at origin surrounded by five equally spaced symbols which in turn are surrounded by six equally spaced symbols. However, there is a difference in constellation diagrams of the proposed architecture with different physical constraints but the BER(Bit Error Rate) performance observed is same.

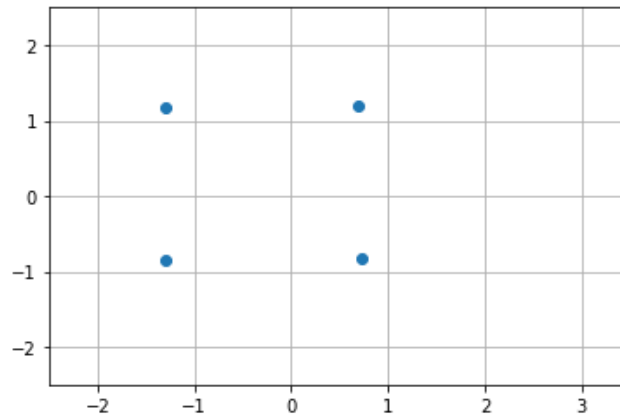


Figure 4.4: Constellation diagram for (2,2) configuration

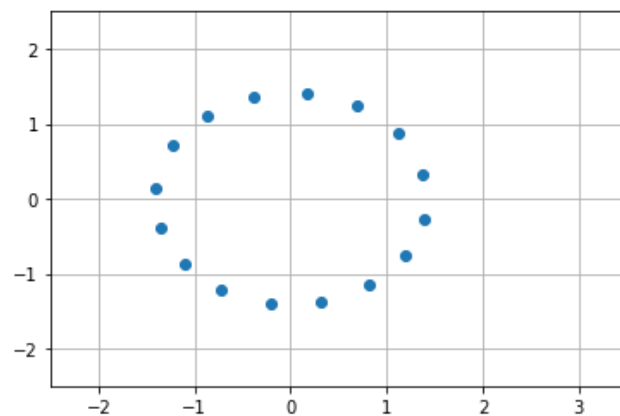


Figure 4.5: Constellation diagram for (2,4) configuration

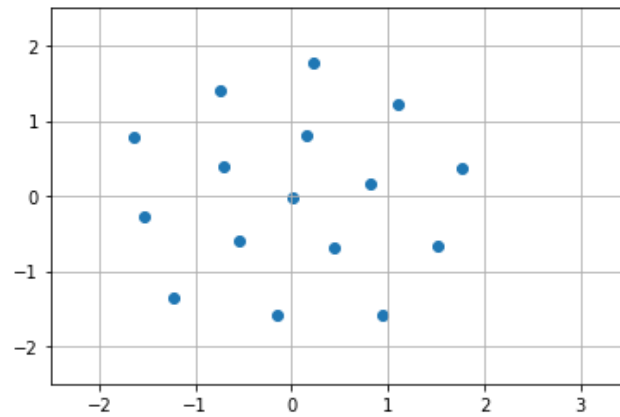


Figure 4.6: Constellation diagram for (2,4) configuration

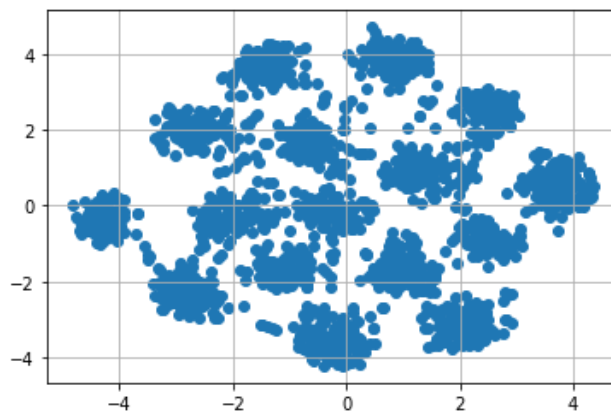


Figure 4.7: Constellation diagram for (7,4) configuration

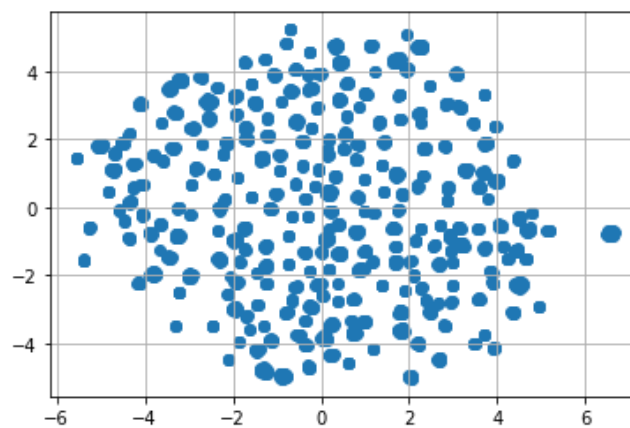


Figure 4.8: Constellation diagram for (8,8) configuration

The last two figures refer to constellation diagrams obtained for higher order autoencoder with more number of channels, figure [4.7] refers to autoencoder configuration (7,4)[21], while figure [4.8] refers to autoencoder (8,8) configuration. These two constellations were observed after the encoded representation of each message was subjected to channel noise, which makes them different from the previously reported constellations. Additionally, t-distributed stochastic neighbour embedding is imposed for the visualization of encoded multi-dimensional noise added representation. It is a non-linear dimensionality reduction technique used to visualize high-dimensional data in low-dimensional space by keeping very similar points close together.

CONCLUSION AND FUTURE WORK

Conventional block structured communication system architectures based on complex mathematical algorithms and statistical theories have gained immense success in the communication sector. But, advancements in technology, increasing size of datasets, and demand for ultra-high-rate transmission pose significant challenges for conventional communication systems. Reliable data transfer over complex channel conditions with the minimum possible delays are among the issues that need to be overcome for the conventional block structured communication systems. However, having a separate block for performing each specific task allows for the analysis and optimization of each block. But, still existing communication systems fail to achieve optimized end-to-end performance in complex scenarios. On the other hand, due to their adaptability to complex scenarios and proven learning abilities with low computational cost, deep learning concepts for communication systems have been increasingly popular to achieve optimized end-to-end learning performance across communication systems. End-to-end learning of communication systems is a novel invention that aims to jointly optimize the transmitter and receiver in an end-to-end manner substituting the need for a conventional block structured communication system. A deep learning-based autoencoder is designed and implemented in four distinct configurations to compare its performance with traditional BPSK configuration over the AWGN channel. Autoencoder configurations were trained for the different number of channel uses and communication rates, and for each of the configurations, BER performance is plotted against the range of $(\mathbb{E}_B/N_0)$ for the test set. In a comparison of the autoencoder-based configurations with corre-

sponding conventional uncoded BPSK configurations, it is found that autoencoder systems have equal or better performance than the traditional communication systems model. Hence, this gives evidence of the learning ability of deep neural networks for achieving end-to-end performance. Moreover, the proposed architecture observes improved BER performance for higher order configurations where channel uses and length of message increases, providing more freedom and learning ability to the model. The learning ability of deep learning models gives them superiority over conventional block structured models. In this work, autoencoder configurations were trained over the fixed value of $(E_B/N_0) = 7\text{db}$, but in the testing phase model was observed to perform well over the given range of (E_B/N_0) from -4 to 12 dB. Additionally, it shows the flexibility of deep learning models to be adaptable to different transmission conditions. This flexibility of deep learning-based models is counted as an advantage over the current traditional systems, which always employ fixed conditions. In addition, because of their reduced computing costs and parallel structure, deep learning models perform better in applications that need low latency and high-rate transmissions. Hence, it can be concluded that Deep learning-based autoencoder architecture's improved learning capabilities and adaptability, lower computational cost and power consumption suggest that they could eventually replace the traditional communication system architecture currently in use.

Furthermore, the application of deep learning models for the physical layer has opened new ventures for research. In this work, we have scrutinized the behavior of the deep learning-based autoencoder communication system for a single user over the AWGN channel only. This work can be further extended to analyze the performance of deep learning models over the various fading channels. Additionally, in this work, we have presumed ideal conditions where conventional communication system architectures also perform well. However, in ideal conditions, deep learning models have comparable performance to traditional architectures. But, the claim to replace the existing conventional block structured communication system architecture with deep learning models become legitimate after analyzing the performance of deep learning-based models in complex scenarios where existing systems are believed to have poor performance. So, the performance comparison of deep learning models in non-ideal situations is still a gap needed to be filled. Further work can be extended for the large message spaces "**M**" with more bits per symbol for multiple user communication

systems.

However, in the end, it can be concluded that the performance and attributes of the deep learning models show their potential to compete with currently used communication system architectures. However, there are still some critical areas for future investigations that are needed to be covered to eliminate the need for traditional block structured communication systems.

References

- [1] Types of losses. <https://keras.io/api/losses/>.
- [2] Types of optimizers. <https://keras.io/api/optimizers/>.
- [3] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, et al. Tensorflow: large-scale machine learning on heterogeneous distributed systems (2016). *arXiv preprint arXiv:1603.04467*, 52, 2015.
- [4] C. Baral, O. Fuentes, and V. Kreinovich. Why deep neural networks: a possible theoretical explanation. In *Constraint programming and decision making: Theory and applications*, pages 1–5. Springer, 2018.
- [5] X. Cheng, D. Liu, Z. Zhu, W. Shi, and Y. Li. A resnet-dnn based channel estimation and equalization scheme in fbmc/oqam systems. In *2018 10th International Conference on Wireless Communications and Signal Processing (WCSP)*, pages 1–5. IEEE, 2018.
- [6] F. Chollet. *Deep learning mit python und keras: das praxis-handbuch vom entwickler der keras-bibliothek*. MITP-Verlags GmbH & Co. KG, 2018.
- [7] E. Dandıl, M. Çakiroğlu, Z. Ekşi, M. Özkan, Ö. K. Kurt, and A. Canan. Artificial neural network-based classification system for lung nodules on computed tomography scans. In *2014 6th International conference of soft computing and pattern recognition (SoCPaR)*, pages 382–386. Ieee, 2014.
- [8] S. Dörner, S. Cammerer, J. Hoydis, and S. Ten Brink. Deep learning based communication over the air. *IEEE Journal of Selected Topics in Signal Processing*, 12(1):132–143, 2017.
- [9] N. Farsad and A. Goldsmith. Detection algorithms for communication systems using deep learning. *arXiv preprint arXiv:1705.08044*, 2017.

- [10] I. Goodfellow, Y. Bengio, and A. Courville. *Deep learning*. MIT press, 2016.
- [11] B. Jdid, K. Hassan, I. Dayoub, W. H. Lim, and M. Mokayef. Machine learning based automatic modulation recognition for wireless communications: A comprehensive survey. *IEEE Access*, 9:57851–57873, 2021.
- [12] Y. Jia, E. Shelhamer, J. Donahue, S. Karayev, J. Long, R. Girshick, S. Guadarrama, and T. Darrell. Caffe: Convolutional architecture for fast feature embedding. In *Proceedings of the 22nd ACM international conference on Multimedia*, pages 675–678, 2014.
- [13] Y. Jiang, H. Kim, H. Asnani, S. Kannan, S. Oh, and P. Viswanath. Turbo autoencoder: Deep learning based channel codes for point-to-point communication channels. *Advances in neural information processing systems*, 32, 2019.
- [14] J. Krause. Study on scenarios and requirements for next generation access technology. 3GPP TR 38.913, Sept, Tech. Rep., 2016.
- [15] J. Liu, K. Mei, X. Zhang, D. Ma, and J. Wei. Online extreme learning machine-based channel estimation and equalization for ofdm systems. *IEEE Communications Letters*, 23(7):1276–1279, 2019.
- [16] A. V. Makkuva, X. Liu, M. V. Jamali, H. MahdaviFar, S. Oh, and P. Viswanath. Ko codes: inventing nonlinear encoding and decoding for reliable wireless communication via deep-learning. In *International Conference on Machine Learning*, pages 7368–7378. PMLR, 2021.
- [17] I. Masood and A. Hassan. Issues in development of artificial neural network-based control chart pattern recognition schemes. *European Journal of Scientific Research*, 39(3): 336–355, 2010.
- [18] D. Neumann, T. Wiese, and W. Utschick. Learning the mmse channel estimator. *IEEE Transactions on Signal Processing*, 66(11):2905–2917, 2018.
- [19] T. J. O’Shea, K. Karra, and T. C. Clancy. Learning to communicate: Channel auto-encoders, domain specific regularizers, and attention. In *2016 IEEE International Symposium on Signal Processing and Information Technology (ISSPIT)*, pages 223–228. IEEE, 2016.

- [20] O. Ozan, G. T. Yamamoto, and U. Demiray. Mobile learning technologies and educational applications. *Mobile Learning Technologies And Educational Applications*, 9(3-4):97–109, 2015.
- [21] T. Oâshea and J. Hoydis. An introduction to deep learning for the physical layer. *IEEE Transactions on Cognitive Communications and Networking*, 3(4):563–575, 2017.
- [22] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.
- [23] T. S. Rappaport, G. R. MacCartney, M. K. Samimi, and S. Sun. Wideband millimeter-wave propagation measurements and channel models for future wireless communication system design. *IEEE transactions on Communications*, 63(9):3029–3056, 2015.
- [24] D. E. Rumelhart, G. E. Hinton, and R. J. Williams. Learning representations by back-propagating errors. *nature*, 323(6088):533–536, 1986.
- [25] J. Shi, S. Hong, C. Cai, Y. Wang, H. Huang, and G. Gui. Deep learning-based automatic modulation recognition method in the presence of phase offset. *IEEE Access*, 8:42841–42847, 2020.
- [26] T. Wang, C.-K. Wen, H. Wang, F. Gao, T. Jiang, and S. Jin. Deep learning for wireless physical layer: Opportunities and challenges. *China Communications*, 14(11):92–111, 2017.
- [27] M. D. Wong and A. K. Nandi. Automatic digital modulation recognition using artificial neural network and genetic algorithm. *Signal Processing*, 84(2):351–365, 2004.
- [28] P. Yadav, S. Kumar, and R. Kumar. A comprehensive survey of physical layer security over fading channels: Classifications, applications, and challenges. *Transactions on Emerging Telecommunications Technologies*, 32(9):e4270, 2021.
- [29] M. Yang, B. Ai, R. He, C. Shen, M. Wen, C. Huang, J. Li, Z. Ma, L. Chen, X. Li, et al. Machine-learning-based scenario identification using channel characteristics in intelligent vehicular communications. *IEEE Transactions on Intelligent Transportation Systems*, 22(7):3961–3974, 2020.

-
- [30] H. Ye, G. Y. Li, and B.-H. Juang. Power of deep learning for channel estimation and signal detection in ofdm systems. *IEEE Wireless Communications Letters*, 7(1):114–117, 2017.
- [31] C. Zhang, P. Patras, and H. Haddadi. Deep learning in mobile and wireless networking: A survey. *IEEE Communications surveys & tutorials*, 21(3):2224–2287, 2019.



Appendix

A.1 Neural Networks

A neural network is the type of architecture used in deep learning models that are inspired by the human brain and mimic how biological neurons communicate with one another in a human being's brain. Multiple layers, divided into input, output, and hidden layers, make up a neural network[10]. The neural network's input layers are used to provide input to the network; these are defined as the first layer of a network, while the output layers are used to obtain output from the neural network. These are also called the last layer or final layer. Layers defined as hidden layers are those that are defined between the input and output layers; as the name suggests, output from each of these layers is not displayed and is hidden inside a network. The collaboration of these layers makes up a neural network which is required to perform the designated job, for example, classification, compression, etc. Whereas, each layer in a neural network approximates some function 'f', which is defined as an activation function that maps input to that layer to the output received from that layer. The purpose of the activation function is to add non-linearity to further enhance the learning ability of the network. For each layer of a neural network, there are defined set of parameters, a set of weights and biases. Weights of each layer are adjusted during training process, to achieve the best possible overall performance over the job designated to neural network. To illustrate, there is a defined input " $\mathbf{x}$ " to the layer with some defined activation function " $\mathbf{f1}$ ", the output from that layer is given by " $\mathbf{y}$ ". " $\mathbf{y}$ " is defined as function approximation " $\mathbf{f1}$ " on

" $\mathbf{x}$ " given by $\mathbf{y} = \mathbf{f}_l(\mathbf{x}; \theta)$, where " θ " is defined as a set of learned parameters of a layer. The types of layers and activation functions used to build a neural network are listed in the tables below.[21].

Name	$f_l(r_{l-1}; \theta_l)$	Parameters
Dense	$\sigma(W_l r_{l-1} + b_l)$	W_l, b_l
Noise	$r_{l-1} + n, n \sim \mathcal{N}(0, \beta I_{N_{l-1}})$	none
Dropout	$\mathbf{d} \cdot r_{l-1}, d_i \sim \text{Bern}(\alpha)$	none
Normalization	e.g , $\frac{\sqrt{N_{l-1} r_{l-1}}}{ r_{l-1} }$	none

Table A.1: Types of Layers of Neural networks

Name	Activation Function $[\sigma(u)]_i$	Range
Linear	u_i	$(-\infty, \infty)$
Relu	$\max(0, u_i)$	$[0, \infty)$
Tanh	$\tanh(u_i)$	$(-1, 1)$
Sigmoid	$\frac{1}{1+e^{-u_i}}$	$(0, 1)$
Softmax	$\frac{e^{u_i}}{\sum_j e^{u_j}}$	$(0, 1)$

Table A.2: Types of Activation Functions

In the tables above, " f_l " represents activation function. Whereas r_l and r_{l-1} are outputs from the current and previous layer, respectively. W_l and b_l are defined as weights and

biases. u_i is the input vector to the layer. Layers can serve various purposes; dense layers are simple, fully connected layers. While layers with stochastic mappings, such as a noise layer, add a randomly generated vector to their input each time they are called. Furthermore, different types of linear and non-linear activation functions are also shown, which enhance the learning capability of the neural networks and make them able to understand complex patterns in the data. Below are mentioned some of the commonly used architectures of the neural networks.

- **Feed forward Neural Networks**

A feed-forward neural network(FNN) or also called multi-layer perceptron (MLP), is a simple neural network that only flows in one direction, it maps input to its output with the collaboration of all the layers involved in the neural network, as the name suggests it is called feed-forward neural network because there is no feedback path drawn from output to the input of the network again and the flow of the network is only in a forward direction from the input to the output side. A feed-forward neural network may consist of single or multiple hidden layers, each with its activation function. In a multiple hidden layers architecture, layers are connected in a manner that the output of a hidden layer is input to the successive layer, and the chain stops at the final layer, which is the final output. Layers are connected to each other through neurons; in a fully connected configuration, each neuron in a layer is connected to all of the neurons of the previous layer. As each neuron is connected to every neuron in the previous layer, the number of parameters of the layer is given by the matrix of order $(k*(p+1))$, where " k " is the number of neurons in the layer and " p " is the number of neurons in the previous layer. The number of neurons in each layer determines the network's width, while the number of layers determines how deep the network operates. Below is the image of a simple, fully connected feed-forward neural network:

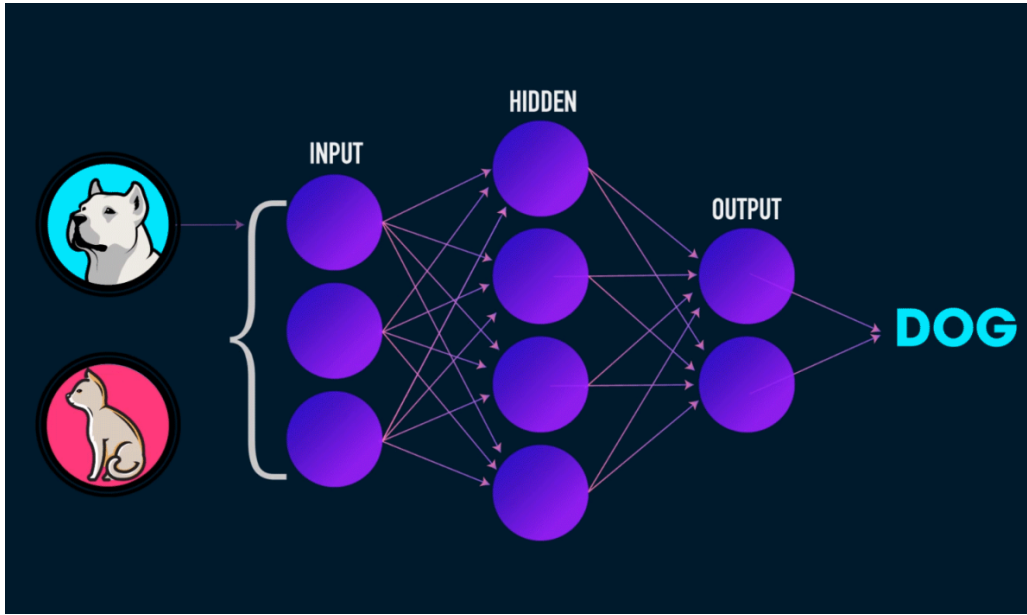


Figure A.1: Fully Connected Feed Forward Neural Network

Figure [A.1] shows that each neuron in the hidden layer is connected to every neuron in the input layer, and each neuron of the hidden layer is connected to each neuron in the output layer in the same way. This highly interconnected arrangement of neurons makes up the neural network. The overall task is defined as a mapping of the input vector to the output vector under some given function " f ". During the training phase, learning algorithms determine the optimal implementation of these hidden layers to achieve optimum end-to-end performance. Each layer of the neural network is made up of parameters termed weights and bias, as was previously discussed. The network's parameters change their values in response to training data while taking into account the activation function of each layer to produce the desired output with minimum possible error[21]. A loss function has been defined to quantify the discrepancy between the achieved and actual output. The network's learning phase seeks to minimize the stated loss function using an optimization technique that iteratively modifies the parameter values before setting the parameter values for which the loss function yields a minimal value. According to the context, various types of loss functions can be defined. Probabilistic losses, regression losses, and hinge losses are the most common types[1]. In the scope of this work[21], we are using categorical cross-entropy as our defined loss function, which falls under the category of probabilistic losses. It computes the cross-entropy losses between predicted and true labels when there are more than

two label classes and input to the network is in the form of a one-hot encoded vector, whereas the output vector represents the probability of each label. To go along with the many types of loss functions that are presented, a variety of optimization methods that are well-suited to those functions are also provided[2]. The goal of an optimizer is to set the values of a parameter of the network, which give the minimum possible value of the defined loss function. The most commonly used optimizer is SGD(Stochastic Gradient Descent) technique. As the name suggests, the SGD method works on minimizing the loss by computing the gradient of the loss function and identifying the global minimum.

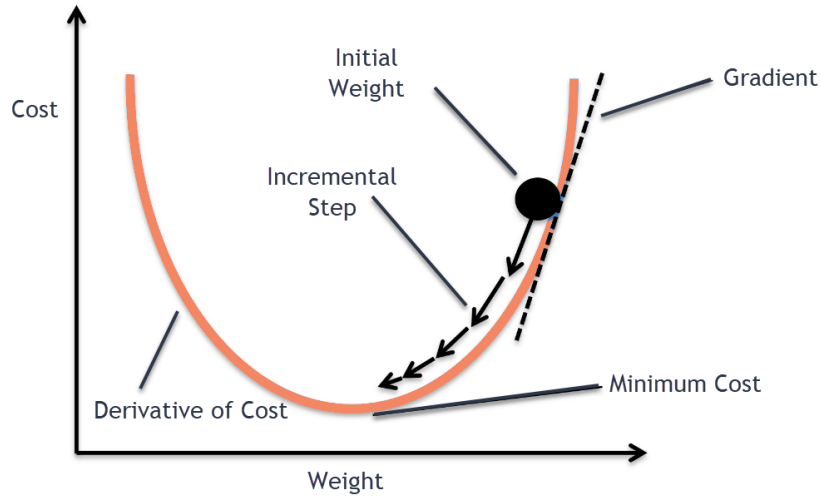


Figure A.2: Gradient Decent technique

As Shown in the figure [A.2] above, the defined loss function is plotted against the values of the parameters of the network. Cost represents the computed loss, whereas weights represent the value of the parameters of the network. In gradient descent, a gradient of the loss function is calculated, and the parameter values are updated iteratively to find the ones that yield the smallest loss.

$$\theta_{t+1} = \theta_t - \eta \nabla \hat{L}(\theta_t) \quad (\text{A.1.1})$$

The above equation [A.1.1] represents how the parameters of network are updated iteratively [21], where $\nabla \hat{L}(\theta_t)$ is defined as gradient of loss function and η is each incremental step after which the gradient is calculated is known as the learning rate. According to above equation, when gradient is positive and it updates the value of

parameters and move from left to right towards the global minimum point. Whereas, when the gradient is negative it moves from right to left to converge at minimum loss.

- **Autoencoders**

The concept of feed-forward neural networks is further prolonged to autoencoders, where multiple layers are stacked together to enable the encoding and decoding process. In an autoencoder, input is first compressed to complex representation through some non-linear activation functions. That compressed representation is then subjected to the combination of multiple layers for decompression to reconstruct the original input. The original aim is to expect the output to be exactly the same as the input, to simulate data transfer from one point to another through a reconstruction job in an unsupervised way. The autoencoder's unique architecture is made in such a way that, rather than memorizing the input data, it learns how to compress and decompress it by introducing non-linearity through activation functions. However, when simulating a communication system, further addition of redundancy in the form of channel noise proved the robustness of autoencoder architectures against the channel perturbations. The figure [A.3] below represents a simple example of the structure and functionality of an autoencoder.

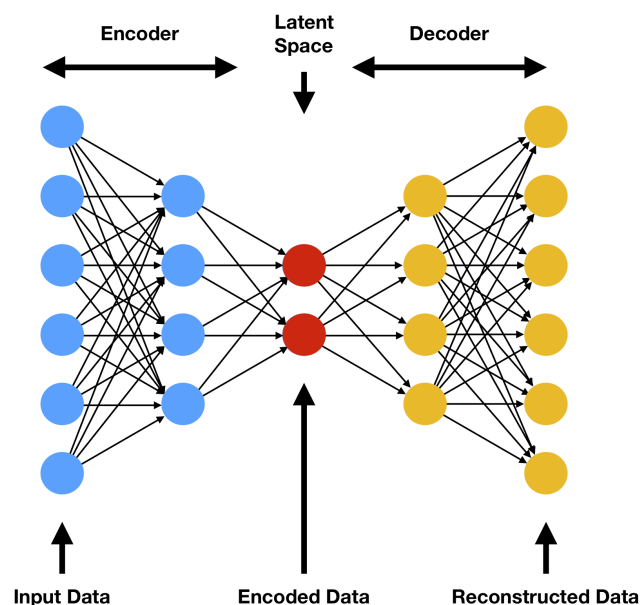


Figure A.3: Autoencoder

Link to GitHub for codes: https://github.com/sa21262/dissertation_codes